SCALE FOR PROJECT RAG AGAINST THE M
(/PROJECTS/RAG-AGAINST-THE-MACHI

You should evaluate 1 student in this team

★

Git repository

```
git@vogsphere.42lyon.fr:vogsphere/intra-uuid-ffee3e7c-8c74-420
```

Introduction

- Remain polite, respectful and constructive during the evaluation.
- This project evaluates information retrieval, text chunking,
answer generation and overall RAG system design.
- Reference exam scripts are provided in `exams/scripts/`:
`exam_retrieval.sh`, `exam_answer.sh`, `exam_edge_cases.sh`
- See `exams/exam_scripts_instructions.md` for detailed corrector guidance.
- The correction is designed so the retrieval pipeline (~8 min) runs
in the background while you inspect code and ask questions.

Guidelines

- Only grade the work present in the student's Git repository.
- Clone the repository in an empty directory.
- Run `cd student && uv sync` before any test.
- The student solution must NEVER import moulinette packages.
Verify: `grep -rn "moulinette" student/ --include="*.py"`
- The program must not crash unexpectedly during evaluation.
- No modification of student code is allowed during correction.
- Unzip private datasets before running exam scripts:
`unzip data/datasets/datasets_private.zip -d data/datasets/`
- Use flags if cheating or plagiarism is detected.

Attachments

- datasets_private.zip (https://cdn.intra.42.fr/document/document/49090/datasets_private.zip)
- exams.zip (https://cdn.intra.42.fr/document/document/49091/exams.zip)
- subject.pdf (https://cdn.intra.42.fr/pdf/pdf/204548/en.subject.pdf)
- datasets_public.zip (https://cdn.intra.42.fr/document/document/49092/datasets_public.zip)
- README.md (https://cdn.intra.42.fr/document/document/49093/README.md)
- vllm-0.10.1.zip (https://cdn.intra.42.fr/document/document/49094/vllm-0.10.1.zip)
- moulinette.zip (https://cdn.intra.42.fr/document/document/49095/moulinette.zip)

Mandatory Part

Preliminaries & Setup

Check the following requirements:

- The Git repository is available and contains only the requested files
- Python 3.10+ is used
- uv is used as the project and package manager
- pyproject.toml and uv.lock files are present
- src/ directory exists with the main module

- README.md file is present
- `cd student && uv sync` completes without errors
- Unzip private datasets: unzip data/datasets/datasets_private.zip -d data/datasets/

Are all setup requirements met?

Yes No

Launch Pipeline (Background)

Launch the automated retrieval pipeline in a background terminal.
This runs indexing, search, and evaluation (~8 min total).

In a separate terminal, run:
./exams/scripts/exam_retrieval.sh \
--student-path ./student \
--moulinette-path ./moulinette-ubuntu

Continue with code inspection (Q3-Q6) while this runs.
You will check the results in Q7a-Q7d.

Did the pipeline launch successfully?

Yes No

Code Quality & Pydantic Models

Review code quality and data model implementation. Check ALL of the following:

Code quality:

- Classes use pydantic for validation and type safety
- Code adheres to flake8 coding standards
- Exceptions are handled gracefully with try-except blocks
- Resources are properly managed (context managers or explicit close())
- Progress bars are implemented for long-running operations (tqdm)

Pydantic models (all 7 must be correctly implemented):

- MinimalSource (file_path, first_character_index, last_character_index)
- UnansweredQuestion and AnsweredQuestion
- RagDataset
- MinimalSearchResults and MinimalAnswer
- StudentSearchResults and StudentSearchResultsAndAnswer

Output format:

- Search operations output valid JSON using StudentSearchResults model
- Answer operations output valid JSON using StudentSearchResultsAndAnswer model
- Each source includes correct file_path, first_character_index, last_character_index

Does the code meet all quality standards and are all models correctly implemented?

Yes No

Chunking Strategies

Verify chunking strategies:

- At least 2 chunking strategies are implemented (e.g., Python code + markdown/text)
- Maximum chunk size is 2000 characters and configurable
- Student can explain the impact of chunk size on retrieval performance: "What happens if chu small? Too large?"

Ask the student to show the code for both strategies.

Are chunking strategies properly implemented?

Yes No

Retrieval System

Verify retrieval implementation:

- At least one method (TF-IDF or BM25) is implemented
- System retrieves and ranks relevant information
- Student can explain how their chosen method works

Live demo: ask the student to run a search query and explain the results:
uv run python -m src search "How to configure OpenAI server?" --k 10

Is the retrieval system properly implemented?

Yes No

Answer Generation

Verify answer generation:

- Qwen/Qwen3-0.6B is used as the DEFAULT model
- Run: uv run python -m src answer "How to configure OpenAI server?" --k 10
- System passes retrieved context to LLM within token limits
- Answers are generated based on retrieved code/documentation

Note: The student may support additional local models beyond the default.
Check the README for documentation of supported models.
During evaluation, always test with the default model (Qwen3-0.6B).

Does answer generation work correctly with the default model?

Yes No

Retrieval Performance: Docs

Check the exam_retrieval.sh results (from Q2 background run).

The script evaluates Recall@5 on the private docs dataset (100 questions).
Look for the "Docs Recall@5" line in the output.

Pass criterion: Docs Recall@5 >= 80%

Note the exact Recall@5 percentage for Q7b star rating.

Does the docs dataset meet the 80% Recall@5 threshold?

Yes No

Retrieval Excellence: Docs

Rate the docs Recall@5 performance using the exact value from Q7a.

Star rating (relative to 80% base threshold):

- 1 star: Q7a failed (below 80%)
- 2 stars: >= 80% (minimum pass)
- 3 stars: >= 85%
- 4 stars: >= 90%
- 5 stars: >= 95%

The exam_retrieval.sh output includes the exact percentage
and a star rating reference table.

Rate it from 0 (failed) through 5 (excellent)

Retrieval Performance: Code

Check the exam_retrieval.sh results (from Q2 background run).

The script evaluates Recall@5 on the private code dataset (100 questions).
Look for the "Code Recall@5" line in the output.

Pass criterion: Code Recall@5 >= 50%

Note the exact Recall@5 percentage for Q7d star rating.

Does the code dataset meet the 50% Recall@5 threshold?

Yes No

Retrieval Excellence: Code

Rate the code Recall@5 performance using the exact value from Q7c.

Star rating (relative to 50% base threshold):

- 1 star: Q7c failed (below 50%)
- 2 stars: >= 50% (minimum pass)
- 3 stars: >= 55%
- 4 stars: >= 60%
- 5 stars: >= 65%

The exam_retrieval.sh output includes the exact percentage
and a star rating reference table.

Rate it from 0 (failed) through 5 (excellent)

Answer Quality

Run the answer exam script to evaluate answer quality:

./exams/scripts/exam_answer.sh \
--student-path ./student \
--moulinette-path ./moulinette-ubuntu

The script lists questions where sources were correctly retrieved,
then you pick 3 questions. For each, the student's answer is displayed.

A satisfactory answer meets ALL of the following:

- Self-contained: readable without seeing the original question
- Source-grounded: cites the source(s) it draws from
- Faithful: limits itself to source content (no hallucination)
- Relevant: directly answers the question asked

Pass if at least 2 out of 3 answers are satisfactory.

Does the student pass the answer evaluation (2/3 answers ok)?

Yes No

Answer Excellence

Rate based on how many of the 3 answers meet all 4 quality criteria
(self-contained, source-grounded, faithful, relevant):

- 1 star: 0/3 answers meet criteria
- 2 stars: 1/3 answers meet criteria
- 3 stars: 2/3 answers meet criteria
- 4 stars: 3/3 answers meet criteria
- 5 stars: 3/3 answers meet criteria with notable depth and thoroughness

Rate it from 0 (failed) through 5 (excellent)

Student Understanding

Test the student's understanding of RAG concepts and their implementation.
Ask these 5 questions (in order):

1. "What is Retrieval-Augmented Generation (RAG) and why is it useful?"
2. "Walk me through your complete RAG pipeline, from raw documents to a generated answer.
3. "What is the difference between TF-IDF and BM25 as retrieval methods?"
4. "What implementation choices did you make and why? What trade-offs did you consider?"
5. "If you had more time, what would you improve in your system?"

Pass requires 4 out of 5 correct/satisfactory answers.
A satisfactory answer demonstrates genuine understanding,
not just memorized definitions.

Does the student demonstrate good understanding (4/5)?

Yes No

README & Documentation

Verify README.md contains all required sections. Checklist:

1. [ ] System architecture (RAG pipeline components and interactions)
2. [ ] Chunking strategy explanation
3. [ ] Retrieval method details (algorithm and ranking mechanism)
4. [ ] Performance analysis (recall@k scores and system performance)
5. [ ] Design decisions and trade-offs
6. [ ] Example usage with clear instructions

If the student implemented bonus features, the README should also document those experiments and their results.

Pass if at least 5 out of 6 items are present and well-documented.

Is the README comprehensive and well-documented?

Yes No

System Reliability

Run the edge case exam script:

./exams/scripts/exam_edge_cases.sh --student-path ./student

This tests 4 degenerate inputs:

1. Empty query: search "" --k 10
2. Gibberish query: search "asdfghjkl" --k 10
3. k=0: answer "What is vLLM?" --k 0
4. Bad dataset path: search_dataset --dataset_path /nonexistent.json

Pass criterion: ALL 4 tests complete without Python tracebacks.
The program may print error messages or return empty results — that is acceptable. Unhandled exceptions with tracebacks are NOT acceptable.

Does the system handle all edge cases without crashing?

Yes No

Bonus Part

Bonus: Advanced RAG

Check for bonus features. Each feature is worth 1 or 2 points.
Sum the points and cap at 5.

Worth 1 point each:

- Query expansion implemented (e.g., synonym expansion, query rewriting)
- Semantic embeddings used for retrieval
- Caching implemented (index caching, query caching, etc.)

Worth 2 points each:

- Hybrid retrieval combining BM25 with semantic embeddings
- LLM inference done using vLLM (local serving)

The corrector may consider other bonus contributions if well justified by the student.

Ask the student to demonstrate each claimed feature.
Verify the feature is actually implemented and working, not just described in the README.

Rate it from 0 (failed) through 5 (excellent)

Ratings

Don’t forget to check the flag corresponding to the defense

✔ Ok ★ Outstanding project

Empty work Incomplete work Invalid compilation Norme Cheat

Concerning situation Forbidden function Can’t support / exp

Conclusion

Leave a comment on this evaluation (2048 chars max)

Finish evaluation

API General Terms of Use (https://profile.intra.42.fr/legal/terms/33)
Declaration on the use of cookies (https://profile.intra.42.fr/legal/terms/2)
Privacy policy (https://profile.intra.42.fr/legal/terms/5)
General term of use of the site (https://profile.intra.42.fr/legal/terms/6)
Rules of pro (https://profile.intra.42